

Rapport TP1 — Docker

Alban Talagrand

2026-06-08

Table des matières

0.1	Base de données	1
0.2	Persistance des données	3
0.3	Backend API	3
0.4	Http server	6
0.5	Link application — docker-compose	9
0.6	Publish — Docker Hub	10

0.1 Base de données

Mon Dockerfile et les scripts SQL d'init sont dans le dossier db/.

Après avoir créé mon Dockerfile, je build l'image en utilisant db/ comme contexte de build, puis je lance le conteneur :

```
+ TP1 git:(main) * docker build -t nabal7/myfirstapp ./db
[+] Building 0.8s (6/6) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 133B
=> [internal] load metadata for docker.io/library/postgres:17.2-alpine
=> [auth] library/postgres:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> CACHED [1/1] FROM docker.io/library/postgres:17.2-alpine@sha256:7e5df973a74872482e320dcdb055e178d6f42de0558b083892c50cda833c96
=> exporting image
=> => exporting layers
=> writing image sha256:8386de4e9c69d40b4a02323b0ea0d530215c81ddec68124e809b1a2113f027e
=> => naming to docker.io/nabal7/myfirstapp

1 warning found (use docker --debug to expand):
- SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "POSTGRES_PASSWORD") (line 3)

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/o34amifk0hroghv3lzetzsbg
+ TP1 git:(main) * docker run -p 8888:5000 --name myfirstapp nabal7/myfirstapp
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.
```

FIGURE 1 – Build de l'image db et lancement du conteneur

Je crée le réseau et je relance le conteneur en attachant le réseau :

```
→ TP1 git:(main) * docker rm myfirstapp
myfirstapp
→ TP1 git:(main) * docker network create app-network
afc54cb2c5ead07d04fd61a80927e472dad938290088c4fc6df177bf8d300ed5
→ TP1 git:(main) * docker run -p 8888:5000 --name myfirstapp --network app-network nabal7/myfirstapp
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.
```

FIGURE 2 – Création du réseau et conteneur attaché au réseau

Question 1-1 Pour quelle raison vaut-il mieux lancer le conteneur avec un flag -e pour donner les variables d'environnement plutôt que de les mettre directement dans le Dockerfile ?

C'est mieux de les mettre dans le flag -e car cela permet de ne pas les exposer dans le Dockerfile, ce qui est plus sécurisé et ça permet de les changer facilement sans avoir à reconstruire l'image.

Une variable ENV est gravée dans l'image et committée dans git, donc un mot de passe en clair reste exposé.

J'ai retiré toutes les variables du Dockerfile :

```
FROM postgres:17.2-alpine
```

```
# Scripts d'init copiés depuis db/docker-entrypoint-initdb.d/  
COPY docker-entrypoint-initdb.d/ /docker-entrypoint-initdb.d/
```

Je passe toutes les variables au run :

```
→ TP1 git:(main) × docker rm myfirstapp  
myfirstapp  
→ TP1 git:(main) × docker run -e POSTGRES_DB=db -e POSTGRES_USER=usr -e POSTGRES_PASSWORD=pwd nabal7/myfirstapp  
The files belonging to this database system will be owned by user "postgres".  
This user must also own the server process.
```

FIGURE 3 – Lancement du conteneur avec les variables d'environnement (-e)

Il faut ajouter le network pour adminer :

```
→ TP1 git:(main) × docker run \
  -p "8090:8090" \
  --net=app-network \
  --name=adminer \
  -d \
  adminer  
Unable to find image 'adminer:latest' locally  
latest: Pulling from library/adminer  
d17f077ada11: Already exists  
fb9121939c47: Pull complete  
0407b1b21ffe: Pull complete  
a764f7935bea: Pull complete  
c20bf0d8e83c: Pull complete  
1330a0ded001: Pull complete  
d5b1a3768b6f: Pull complete  
6c3fe705c2c1: Pull complete  
9c5410126fb7: Pull complete  
78410acc35b3: Pull complete  
99908a2d2aac: Pull complete  
0f5f1cc0d053: Pull complete  
4f4fb700ef54: Pull complete  
dd4bbe85a28b: Pull complete  
75de33216829: Pull complete  
8a62a12d2ec7: Pull complete  
0bd3e93b9145: Pull complete  
Digest: sha256:832668141a338ba93dd7bbf5ee4f5f351d98088b959719cf82b2bca785c7e08c  
Status: Downloaded newer image for adminer:latest  
2d8dc5b74c984cb6e3dbfd169c08d182414b3abf1f75b83c8cafcfe608e5e11  
→ TP1 git:(main) × docker run -p 8888:5000 --name myfirstapp --network app-network -e POSTGRES_DB=db -e POSTGRES_USER=usr -e POSTGRES_PASSWORD=pwd nabal7/myfirstapp
```

FIGURE 4 – Ajout du réseau app-network à adminer

On se connecte ensuite sur adminer

Système	PostgreSQL ▼
Serveur	myfirstapp
Utilisateur	usr
Mot de passe	... 
Base de données	db

☒ Authentification ☐ Authentification permanente

FIGURE 5 – Connexion à la base via adminer

0.2 Persistance des données

Sans volume, détruire le conteneur efface toutes les données. J'attache un volume nommé sur le dossier de données de postgres (/var/lib/postgresql/data) :

```
→ TP1 git:(main) × docker run -d -p 8888:5432 --name myfirstapp --network app-network \
-e POSTGRES_DB=db -e POSTGRES_USER=usr -e POSTGRES_PASSWORD=pwd \
-v db-data:/var/lib/postgresql/data \
nabal7/myfirstapp
1c7f968d5b6017350dc8eeab731d3a17f2dd74fc9c48fd0eddc33dc8a5ffdf85
```

FIGURE 6 – Attachement du volume nommé db-data

Test : je détruis puis recrée le conteneur, les données insérées survivent car elles sont stockées dans le volume db-data sur le disque hôte.

```
→ TP1 git:(main) × docker run -d -p 8888:5432 --name myfirstapp --network app-network \
-e POSTGRES_DB=db -e POSTGRES_USER=usr -e POSTGRES_PASSWORD=pwd \
-v db-data:/var/lib/postgresql/data \
nabal7/myfirstapp
1c7f968d5b6017350dc8eeab731d3a17f2dd74fc9c48fd0eddc33dc8a5ffdf85
→ TP1 git:(main) × docker rm -f myfirstapp
myfirstapp
→ TP1 git:(main) × docker run -d -p 8888:5432 --name myfirstapp --network app-network \
-e POSTGRES_DB=db -e POSTGRES_USER=usr -e POSTGRES_PASSWORD=pwd \
-v db-data:/var/lib/postgresql/data \
nabal7/myfirstapp
c6f91ca03969ed29a651d94cebba9c544fcc62f422415fd8b70cb1fc33c37526
→ TP1 git:(main) × |
```

FIGURE 7 – Persistance des données après destruction/recréation du conteneur

Question 1-2 Pourquoi avons-nous besoin d'attacher un volume à notre conteneur postgres ?

Le conteneur est éphémère : sa couche d'écriture disparaît avec lui. Postgres écrit ses données dans /var/lib/postgresql/data, qui est dans le conteneur. Sans volume, ces données meurent à chaque docker rm.

Question 1-3 Documentez les éléments essentiels de votre conteneur de base de données : commandes et Dockerfile.

Voir db/README.md : arborescence, Dockerfile, commandes essentielles (build, network, run, adminer), inspection et persistance.

0.3 Backend API

0.3.1 Basics — Java Hello World

J'écris un Main.java minimal, je le compile sur l'hôte puis je le fais tourner dans un conteneur.

Dockerfile (le .class est compilé sur l'hôte, l'image n'embarque qu'un JRE) :

```
FROM eclipse-temurin:21-jre-alpine
COPY Main.class .
CMD ["java", "Main"]
```

```

→ TP1 git:(main) × cd backend
→ backend git:(main) × javac --release 21 Main.java
→ backend git:(main) × docker build -t nabal7/hello-java .
[+] Building 1.6s (7/7) FINISHED                                docker:desktop-linux
⇒ [internal] load build definition from Dockerfile                0.0s
⇒ ⇒ transferring dockerfile: 168B                                0.0s
⇒ [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine 1.6s
⇒ [internal] load .dockerignore                                  0.0s
⇒ ⇒ transferring context: 2B                                      0.0s
⇒ [internal] load build context                                  0.0s
⇒ ⇒ transferring context: 453B                                    0.0s
⇒ [1/2] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:704db3c40204a44f4 0.0s
⇒ CACHED [2/2] COPY Main.class .                                0.0s
⇒ exporting to image                                             0.0s
⇒ ⇒ exporting layers                                             0.0s
⇒ ⇒ writing image sha256:1ccaa4739f343d5da711a4a6f153d89522b75f643f953bfa180d2640b13 0.0s
⇒ ⇒ naming to docker.io/nabal7/hello-java                       0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/f5dvejgyhqgx01qbxw14d9zd
→ backend git:(main) × docker run nabal7/hello-java
Hello World!
→ backend git:(main) × |

```

FIGURE 8 – Compilation, build et exécution du Hello World Java

0.3.2 Backend simple api — Springboot (build multistage)

Je génère une appli Springboot (Spring Web, Java 21) avec un endpoint / qui renvoie un JSON Greeting. Au lieu de compiler le .jar sur ma machine, je laisse Docker s'en charger via un **build multistage**.

Dockerfile :

```

# Build stage
FROM eclipse-temurin:21-jdk-alpine AS myapp-build
ENV MYAPP_HOME=/opt/myapp
WORKDIR $MYAPP_HOME
RUN apk add --no-cache maven
COPY pom.xml .
COPY src ./src
RUN mvn package -DskipTests

# Run stage
FROM eclipse-temurin:21-jre-alpine
ENV MYAPP_HOME=/opt/myapp
WORKDIR $MYAPP_HOME
COPY --from=myapp-build $MYAPP_HOME/target/*.jar $MYAPP_HOME/myapp.jar
ENTRYPOINT ["java", "-jar", "myapp.jar"]

```

Build de l'image et démarrage du conteneur :

Appel API :

Question 1-4 Pourquoi a-t-on besoin d'un build multistage ? Expliquez chaque étape de ce Dockerfile.

Le build multistage sépare l'environnement de compilation de l'environnement d'exécution. Sans lui, l'image finale embarquerait le JDK complet + Maven + tout le cache de dépendances et serait trop lourde. Le multistage ne garde dans l'image finale que le .jar et un JRE → image légère et sûre.

[illegible]

FIGURE 9 – Build multistage de l’image Springboot

```
→ backend git:(main) × curl http://localhost:8080/
{"id":4,"content":"Hello, World!"}%
→ backend git:(main) × curl "http://localhost:8080/?name=Alban"
{"id":5,"content":"Hello, Alban!"}%
→ backend git:(main) ×
```

FIGURE 10 – Appel de l'endpoint greeting de l'API

Étapes :

- **Stage myapp-build** (eclipse-temurin:21-jdk-alpine) : contient le JDK pour compiler.
 - ENV MYAPP_HOME + WORKDIR : définit le répertoire de travail.
 - apk add maven : installe Maven.
 - COPY pom.xml puis COPY src : copie le descriptif puis le code source.
 - mvn package -DskipTests : compile et produit le .jar dans target/.
- **Stage final** (eclipse-temurin:21-jre-alpine) : ne contient qu'un JRE pour exécuter.
 - COPY --from=myapp-build .../target/*.jar myapp.jar : récupère uniquement le .jar du stage précédent.
 - ENTRYPOINT ["java", "-jar", "myapp.jar"] : lance l'application.

0.3.3 Backend API connecté à la base de données

Je remplace src + pom.xml par la source simple-api-student (entités JPA Student/Department, DAO, services, controllers). Je garde mon Dockerfile multistage.

Je configure src/main/resources/application.yml pour joindre la DB. La connexion passe par le réseau Docker, donc l'URL utilise le nom du conteneur postgres (myfirstapp) et son port interne 5432 :

```
spring:
  datasource:
    url: jdbc:postgresql://myfirstapp:5432/db
    username: usr
    password: pwd
    driver-class-name: org.postgresql.Driver
```

Le backend et la DB doivent tourner sur le même réseau app-network pour que la résolution par nom fonctionne :

L'API répond et lit bien les données insérées dans postgres (GET /departments/IRC/students) :

0.4 Http server

0.4.1 Basics — landing page

Je pars de l'image httpd:2.4 et je copie une index.html dans le dossier servi par Apache (/usr/local/apache2/htdocs/).

```
FROM httpd:2.4
COPY index.html /usr/local/apache2/htdocs/
```

Je build, lance, et vérifie la page + les commandes d'inspection (docker stats, docker inspect, docker logs) :

0.4.2 Reverse proxy

Je récupère la config Apache par défaut depuis le conteneur :

```
docker cp httpd:/usr/local/apache2/conf/httpd.conf ./httpd.conf
```

J'active les modules proxy (décommentés) et j'ajoute un VirtualHost qui renvoie tout vers le backend (simpleapi:8080, résolu par nom sur app-network) :

```
LoadModule proxy_module modules/mod_proxy.so
LoadModule proxy_http_module modules/mod_proxy_http.so

<VirtualHost *:80>
```



```

docker build -t nabal7/httpd .
[+] Building 4.6s (8/8) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 111B                      0.0s
=> [internal] load metadata for docker.io/library/httpd  1.6s
=> [auth] library/httpd:pull token for registry-1.docker 0.0s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 2B                            0.0s
=> [internal] load build context                        0.0s
=> => transferring context: 250B                          0.0s
=> [1/2] FROM docker.io/library/httpd:2.4@sha256:2a7de 2.5s
=> => resolve docker.io/library/httpd:2.4@sha256:2a7de 0.0s
=> => sha256:87e0a607bc380c4e183acac20 8.00kB / 8.00kB 0.0s
=> => sha256:cda3d70ae7d7c3d0b3b57a9 30.14MB / 30.14MB 0.9s
=> => sha256:5cd98e6270b027247cbc4bf3827a9 146B / 146B 0.5s
=> => sha256:4f4fb700ef54461cfa02571ae0db9a0 32B / 32B 0.5s
=> => sha256:2a7deaaaf357261a1dffcb8 10.14kB / 10.14kB 0.0s
=> => sha256:7d0c026b015aac804d96aa1fe 2.10kB / 2.10kB 0.0s
=> => sha256:3c49909278fd6ca2279feda4d 1.97MB / 1.97MB 1.0s
=> => sha256:f4e573e52b169e28e70d7cf 13.39MB / 13.39MB 1.3s
=> => extracting sha256:cda3d70ae7d7c3d0b3b57a99a2085f 1.0s
=> => sha256:22e73cca7d143f37984dfa041b8e6 292B / 292B 1.1s
=> => extracting sha256:5cd98e6270b027247cbc4bf3827a96 0.0s
=> => extracting sha256:4f4fb700ef54461cfa02571ae0db9a 0.0s
=> => extracting sha256:3c49909278fd6ca2279feda4d87f9b 0.1s
=> => extracting sha256:f4e573e52b169e28e70d7cfffcae381 0.3s
=> => extracting sha256:22e73cca7d143f37984dfa041b8e60 0.0s
=> [2/2] COPY index.html /usr/local/apache2/htdocs/    0.3s
=> => exporting to image                                0.0s
=> => exporting layers                                  0.0s
=> => writing image sha256:966fce54570d372a8a154dd44d4 0.0s
=> => naming to docker.io/nabal7/httpd                  0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/z39p5gvtn4v4seei1coqo01lg
→ http-server git:(main) x docker run -d -p 8081:80 --name httpd --network app-network nabal7/httpd
4ee08ad27c122116503b07473e786fde1c7012202bb43c195b808c880fd1d3ac
→ http-server git:(main) x curl http://localhost:8081/
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>TP1 - HTTP Server</title>
</head>
<body>
  <h1>Hello from Apache httpd</h1>
  <p>Serveur HTTP du TP1 DevOps.</p>
</body>
</html>
→ http-server git:(main) x docker stats --no-stream
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O     PIDS
4ee08ad27c12   httpd     0.01%     6.363MiB / 7.654GiB  0.08%     3.46kB / 2.2kB  0B / 4.1kB    82
98d968cd08d7   simpleapi 0.56%     277.6MiB / 7.654GiB  3.54%     19.3kB / 20.5kB 0B / 262kB    45
f9d28b069fd6   myfirstapp 0.00%     33.64MiB / 7.654GiB  0.43%     19.6kB / 16.5kB 0B / 225kB    16
2d8dc5b74c98   adminer   0.00%     13.21MiB / 7.654GiB  0.17%     475kB / 718kB   0B / 217kB    1

```

FIGURE 13 – Landing page Apache et commandes d'inspection


```
ProxyPreserveHost On
ProxyPass / http://simpleapi:8080/
ProxyPassReverse / http://simpleapi:8080/
</VirtualHost>
```

Je copie cette config dans l'image (COPY httpd.conf /usr/local/apache2/conf/httpd.conf), rebuild, et l'API répond maintenant via Apache sur le port 80 :

```
+ http-server git:(main) x docker rm -f httpd
httpd
+ http-server git:(main) x docker build -t nabal7/httpd .
[+] Building 1.3s (9/9) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 245B                             0.0s
=> [internal] load metadata for docker.io/library/httpd:2.4     1.2s
=> [auth] library/httpd:pull token for registry-1.docker.io     0.0s
=> [internal] load .dockerignore                                 0.0s
=> => transferring context: 2B                                    0.0s
=> [1/3] FROM docker.io/library/httpd:2.4@sha256:2a7deaaaf357261a1dffc88fc725f 0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 21.10kB                             0.0s
=> CACHED [2/3] COPY index.html /usr/local/apache2/htdocs/      0.0s
=> [3/3] COPY httpd.conf /usr/local/apache2/conf/httpd.conf     0.0s
=> exporting to image                                           0.0s
=> => exporting layers                                           0.0s
=> writing image sha256:76d62f96085902c9f798be288ce56610b53ddc528ca8c43cae0 0.0s
=> naming to docker.io/nabal7/httpd                             0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/vn40yktzf37ibp10i35iycb06
+ http-server git:(main) x docker run -d -p 80:80 --name httpd --network app-network nabal7/httpd
e51f72aca8d17c72a83ddb5e106ba07d0e705bf9872d6f880d2242a0dbc107fc
+ http-server git:(main) x curl http://localhost/departments/IRC/students
[{"id":1,"firstname":"Eli","lastname":"Copter","department":{"id":1,"name":"IRC"}}]
+ http-server git:(main) x docker logs httpd
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.18.0.5. Set the 'ServerName' directive globally to suppress this message
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.18.0.5. Set the 'ServerName' directive globally to suppress this message
[Mon Jun 08 14:57:36.727951 2026] [mpm_event:notice] [pid 1:tid 1] AH00489: Apache/2.4.67 (Unix) configured -- resuming normal operations
+ http-server git:(main) x
```

FIGURE 14 – API accessible via le reverse proxy Apache (port 80)

Question 1-5 Pourquoi a-t-on besoin d'un reverse proxy ?

Le reverse proxy est un point d'entrée unique placé devant l'application. Il masque l'architecture interne : le client ne parle qu'à Apache (port 80), le backend n'est jamais exposé directement. Il permet aussi de gérer au même endroit le SSL/TLS, le load balancing entre plusieurs instances, le cache, la compression, et de servir un front-end statique. On découple ainsi l'accès public de l'organisation interne des conte-neurs.

0.5 Link application — docker-compose

Lancer/arrêter/rebuild les 3 conteneurs à la main est pénible. Je les orchestre avec un seul docker-compose.yml à la racine de TP1/.

Choix de conception :

- **Seul httpd expose un port (80:80).** Le backend et la base n'ont aucun port mappé sur l'hôte → joignables uniquement via le réseau interne, donc plus sûr.
- Identifiants postgres dans un fichier .env (gitignoré), injectés en variables d'environnement — jamais en dur dans un fichier versionné.
- Le backend reçoit sa datasource via SPRING_DATASOURCE_* (override de application.yml).
- healthcheck pg_isready sur la base + depends_on: condition: service_healthy sur le backend → il démarre seulement quand postgres est réellement prêt (et pas juste "lancé").
- restart: unless-stopped sur les 3 services.
- Un réseau app-network partagé + un volume nommé db-data pour la persistance.

```
docker compose up --build -d    # build + démarrage des 3 services
docker compose ps               # vérifie l'état (db healthy, pas de port sur backend/db)
```

L'application complète répond via http://localhost/ (httpd → backend → base) :

```
[+] up 8/8
✓ Image tp1-backend      Built          3.7s
✓ Image tp1-httpd        Built          3.7s
✓ Image tp1-database      Built          3.7s
✓ Network tp1_app-network Created         0.0s
✓ Volume tp1_db-data      Created         0.0s
✓ Container myfirstapp    Healthy        5.7s
✓ Container simpleapi     Started        5.8s
✓ Container httpd         Started        5.8s
→ TP1 git:(main) ✗ docker compose ps      # les 3 services up, healthy
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS          PORTS
httpd                tp1-httpd            "httpd-foreground"    httpd      7 seconds ago  Up 1 second    0.0.0.0:80→80/tcp
myfirstapp           tp1-database         "docker-entrypoint.s..." database    7 seconds ago  Up 7 seconds (healthy)  5432/tcp
simpleapi             tp1-backend          "java -jar myapp.jar" backend     7 seconds ago  Up 1 second
```

```
→ TP1 git:(main) ✗ curl http://localhost/departments/IRC/students
→ TP1 git:(main) ✗ curl http://localhost/departments/IRC/students
[{"id":1,"firstname":"Eli","lastname":"Copter","department":{"id":1,"name":"IRC"}}]
→ TP1 git:(main) ✗
```

FIGURE 15 – Application complète orchestrée via docker-compose

Question 1-6 Pourquoi docker-compose est-il si important ?

Docker-compose décrit toute l'infrastructure multi-conteneurs dans un seul fichier déclaratif versionné. Au lieu de taper et coordonner manuellement une dizaine de `docker build/docker run/docker network/docker volume`, une seule commande monte ou détruit toute la stack, dans le bon ordre (grâce à `depends_on`). C'est reproductible, partageable dans le repo, et c'est de l'infrastructure-as-code : tout le monde lance exactement le même environnement.

Question 1-7 Documentez les commandes docker-compose les plus importantes.

- `docker compose up -d` : démarre tous les services en arrière-plan.
- `docker compose up --build -d` : rebuild les images avant de démarrer.
- `docker compose ps` : liste l'état des services.
- `docker compose logs -f <service>` : suit les logs d'un service.
- `docker compose stop / start` : arrête / relance sans détruire.
- `docker compose down` : arrête et supprime conteneurs + réseau (`-v` pour aussi le volume).
- `docker compose build` : (re)build les images sans démarrer.

0.6 Publish — Docker Hub

J'ai ajouté un champ `image` : à chaque service du compose pour qu'elles soient buildées directement avec le bon nom et un tag de version (`:1.0`) au lieu de `latest`.

```
docker login                # connexion au compte Docker Hub
docker compose build        # build des images nabal7/tp1-*:1.0
docker push nabal7/tp1-database:1.0
docker push nabal7/tp1-backend:1.0
docker push nabal7/tp1-httpd:1.0
```

Les 3 images apparaissent ensuite dans mon compte Docker Hub :

Question 1-10 Pourquoi mettre nos images dans un dépôt en ligne ?

Un dépôt en ligne sert de source partagée et centralisée : les coéquipiers et les autres machines (serveurs, CI/CD, prod) récupèrent l'image exacte avec `docker pull` au lieu de la rebuild. Le tag de version garantit que tout le monde déploie le même artefact, et l'image devient déployable n'importe où sans recompiler le code.

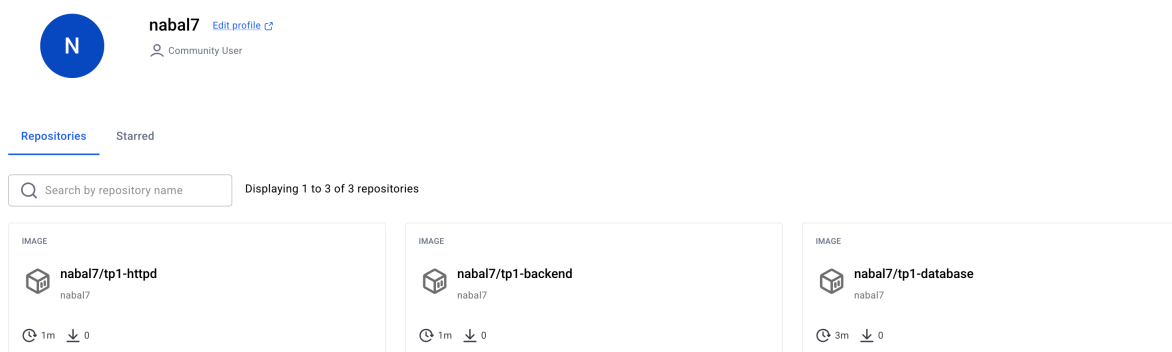


FIGURE 16 – Images publiées sur Docker Hub

Rapport TP2 — GitHub Actions (CI/CD)

Alban Talagrand

Table des matières

TP2 — GitHub Actions (CI/CD)	1
Étape 1 — Première CI : build + test du backend	1
2-1 — Que sont les testcontainers?	2
Étape 2 — CD : build + push des images Docker	2
2-2 — Pourquoi utiliser des variables sécurisées?	3
2-3 — Pourquoi needs: test-backend sur ce job?	3
2-4 — Pourquoi pousser les images Docker?	3
Étape 3 — Quality Gate : SonarCloud	4
Étape 4 — Aller plus loin : séparation des pipelines	4

TP2 — GitHub Actions (CI/CD)

Dépôt utilisé : tp-devops-correction-docker (forké au TD2), distant `git@github.com:Nabal22/tp-devops-correction-docker`

Étape 1 — Première CI : build + test du backend

J'ai créé `.github/workflows/main.yml` avec un job `test-backend` qui : 1. récupère le code (`actions/checkout@v4`), 2. installe le JDK 21 (`actions/setup-java@v4`, distribution `temurin`), 3. construit et teste l'application avec `mvn clean verify --file ./simple-api/pom.xml`.

Le pipeline se déclenche sur `push` (branches `main` et `develop`) et sur `pull_request`.

`name: CI devops 2025`

`on:`

`# Lancer le pipeline sur les branches main et develop`

`push:`

`branches:`

`- main`

`- develop`

`pull_request:`

`jobs:`

`test-backend:`

`runs-on: ubuntu-24.04`

`steps:`

`# Récupérer le code du dépôt`

`- name: Checkout code`

`uses: actions/checkout@v4`

`# Installer le JDK 21 (distribution Temurin)`

`- name: Set up JDK 21`

```

uses: actions/setup-java@v4
with:
  distribution: temurin
  java-version: '21'

# Construire l'application et lancer les tests (unitaires + intégration)
- name: Build and test with Maven
  run: mvn clean verify --file ./simple-api/pom.xml

```

← CI devops 2025

✓ ci: add test-backend workflow (build + test with Maven) #1

The screenshot displays the GitHub Actions interface for a workflow named 'test-backend'. On the left, a sidebar shows the workflow's status as 'test-backend' with a green checkmark. Below this, there are links for 'Run details', 'Usage', and 'Workflow file'. The main area on the right shows the workflow's execution details. At the top, it says 'Annotations' with '1 warning'. Below that, the workflow name 'test-backend' is shown with the status 'succeeded 2 minutes ago in 1m 6s'. A list of steps follows, each with a green checkmark indicating success: 'Set up job', 'Checkout code', 'Set up JDK 21', 'Build and test with Maven', 'Post Set up JDK 21', 'Post Checkout code', and 'Complete job'.

FIGURE 1 – CI test-backend verte

2-1 — Que sont les testcontainers ?

Ce sont des bibliothèques Java qui permettent de lancer des conteneurs Docker pendant l'exécution des tests. Ici un conteneur PostgreSQL est démarré automatiquement pour les tests d'intégration, ce qui permet de tester l'accès réel à la base sans dépendre d'une base externe installée. Le conteneur est jetable : créé au début des tests, détruit à la fin.

`mvn clean verify` nettoie les builds précédents, recompile chaque module, puis lance les tests unitaires **et** d'intégration.

Étape 2 — CD : build + push des images Docker

J'ai ajouté un job `build-and-push-docker-image` qui se connecte à Docker Hub puis construit/pousse les 3 images (`simple-api`, `database`, `httpd`). Les identifiants Docker Hub sont stockés dans les **secrets GitHub** (`DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`), jamais en clair dans le YAML.

```

build-and-push-docker-image:
  # S'exécute uniquement si la compilation et les tests sont OK
  needs: test-backend
  runs-on: ubuntu-24.04
  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Login to DockerHub
      run: echo "${{ secrets.DOCKERHUB_TOKEN }}" | docker login --username "${{ secrets.DOCKERHUB_USERNAME }}" --password-stdin

    - name: Build image and push backend
      uses: docker/build-push-action@v6
      with:
        context: ./simple-api
        tags: "${{ secrets.DOCKERHUB_USERNAME }}/tp-devops-simple-api:latest"
        push: "${{ github.ref == 'refs/heads/main' }}"
        # idem pour database (./database) et httpd (./http-server)

```

2-2 — Pourquoi utiliser des variables sécurisées ?

Pour ne jamais exposer les identifiants Docker Hub (username/token) dans le code. Le dépôt étant public, mettre les credentials en clair dans le YAML les rendrait visibles par tout le monde et exploitables. Les secrets GitHub sont chiffrés, injectés à l'exécution via `${{ secrets.NAME }}` et masqués dans les logs.

2-3 — Pourquoi needs: test-backend sur ce job ?

needs crée une dépendance : build-and-push-docker-image ne démarre que si test-backend a réussi. On ne veut pas construire ni publier une image si le code ne compile pas ou si les tests échouent. Sans needs, les deux jobs tourneraient en parallèle et on pourrait publier une image cassée.

2-4 — Pourquoi pousser les images Docker ?

Pour les rendre disponibles sur un registre (Docker Hub) afin qu'elles soient réutilisables et déployables partout, sans avoir à reconstruire le code. C'est l'étape de **livraison continue** (CD) : à chaque commit sur main, une image à jour et testée est publiée et prête à être déployée.

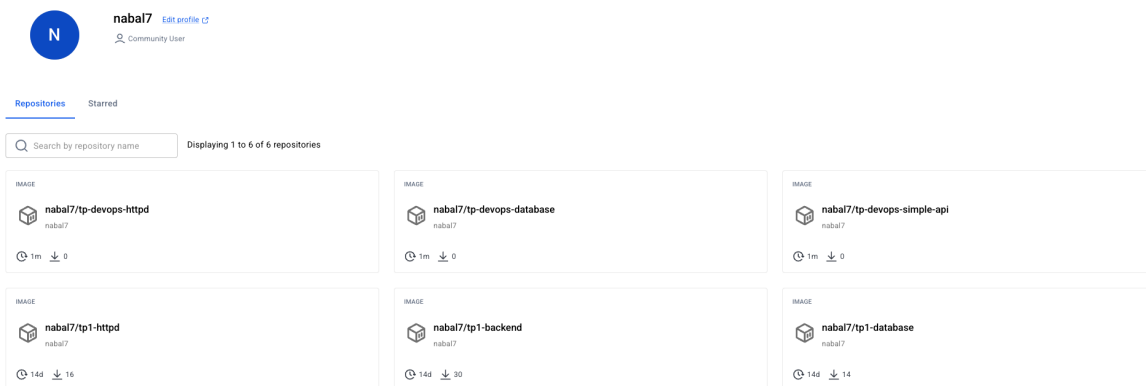


FIGURE 2 — Images sur Docker Hub

✓ ci: build and push 3 docker images on main (needs test-backend) #2

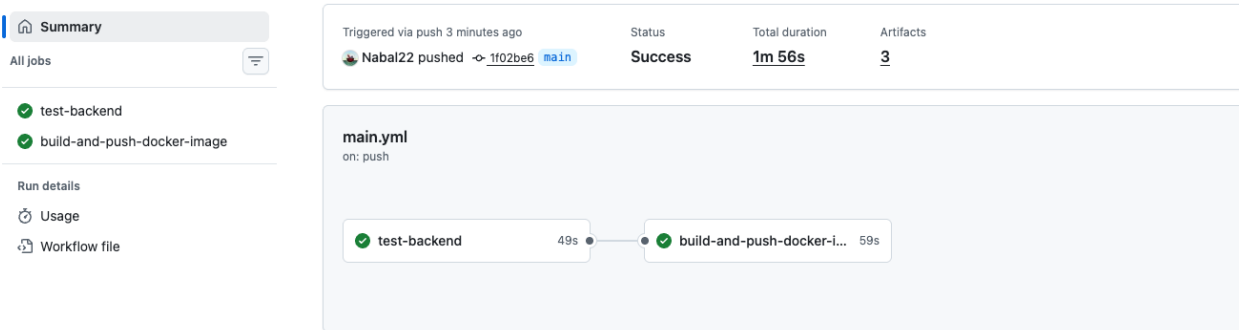


FIGURE 3 – Pipeline complet

Étape 3 — Quality Gate : SonarCloud

J'ai créé un compte SonarCloud, une organisation (tp-devops-alban-genial) et un projet (clé tp). J'ai modifié l'étape Maven pour lancer l'analyse SonarCloud en même temps que le build/test. Le token Sonar est stocké dans le secret GitHub SONAR_TOKEN (jamais en clair). Jacoco génère la couverture de code pendant verify, puis sonar:sonar l'envoi à SonarCloud.

```
- name: Build and test with Maven + SonarCloud
  run: mvn -B verify sonar:sonar -Dsonar.projectKey=tp -Dsonar.organization=tp-devops-alban-genial
```

Étape 4 — Aller plus loin : séparation des pipelines

J'ai séparé le pipeline unique en **deux workflows** distincts :

- test-backend.yml (CI devops 2025 - Test Backend) : build + test + analyse SonarCloud, déclenché sur push (branches main et develop) et sur les pull requests.
- build-and-push.yml (Build and Push Docker Images) : build + push des 3 images, déclenché **uniquement** quand le workflow de test se termine, et seulement sur main.

Le lien entre les deux se fait avec on: workflow_run : le workflow de build n'écoute pas les push, il attend la fin du workflow de test. Le filtre branches: [main] limite le déclenchement à main, et la condition if: github.event.workflow_run.conclusion == 'success' garantit qu'on ne build/push que si les tests sont **verts**.

Résultat : - test-backend → sur develop ET main. - build-and-push-docker-image → sur main uniquement, et seulement si les tests passent.

```
# build-and-push.yml
on:
  workflow_run:
    workflows: ["CI devops 2025 - Test Backend"]
    types:
      - completed
  branches:
    - main

jobs:
  build-and-push-docker-image:
    if: ${ github.event.workflow_run.conclusion == 'success' }}
```

Overview

Private · No tags · 482 Lines of Code · Last analysis 2 minutes ago

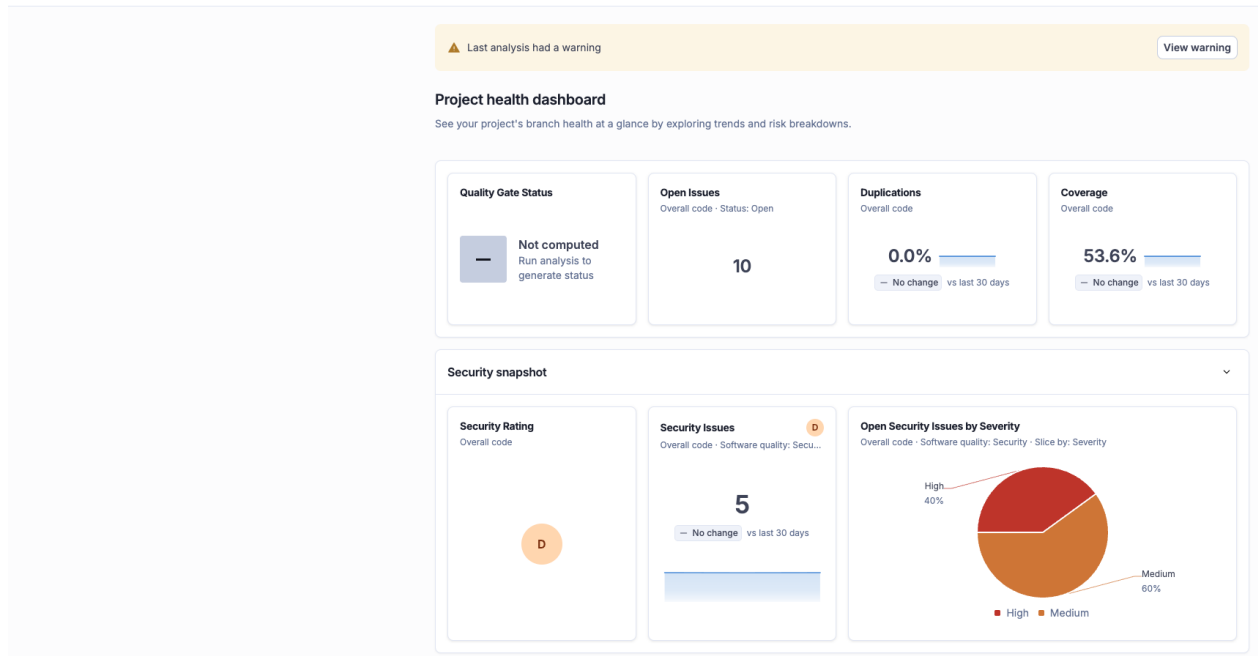


FIGURE 4 – Rapport SonarCloud

runs-on: ubuntu-24.04

... login DockerHub + build/push des 3 images (push: true)

Note : workflow_run ne se déclenche que si le fichier du workflow déclencheur est présent sur la branche par défaut (main). Le tout premier push qui ajoute ces fichiers ne déclenche donc pas encore le build ; c'est le push suivant sur main qui enchaîne test → build/push.

Build and Push Docker Images Build and Push Docker Images #2: completed by Nabal22		3 minutes ago 1m 4s ...
ci: trigger workflow CI devops 2025 - Test Backend #4: Commit d3b005d pushed by Nabal22	develop	3 minutes ago 1m 19s ...
ci: trigger workflow CI devops 2025 - Test Backend #3: Commit d3b005d pushed by Nabal22	main	5 minutes ago 1m 39s ...

FIGURE 5 – Pipelines séparés

Rapport TP3 — Ansible (déploiement et CD)

Alban Talagrand

Table des matières

TP3 — Ansible : déploiement automatisé	1
Étape 1 — Inventaire	1
3-1 — Inventaire et commandes de base	1
Étape 2 — Facts et nettoyage	2
Étape 3 & 4 — Playbook : installation de Docker	2
3-2 — Le playbook	2
Étape 5 — Refactor en rôle docker	2
Étape 6 — Déploiement de l'application (rôles + docker_container)	4
3-3 — Configuration des tâches docker_container	4
Étape 7 — Déploiement continu (GitHub Actions)	4
Question — Est-ce vraiment sûr de déployer automatiquement chaque nouvelle image?	6
Étape 8 — Front-end	7
Étape 9 — Aller plus loin : Ansible Vault	7

TP3 — Ansible : déploiement automatisé

Le répertoire `ansible/` vit dans le dépôt de l'application (`tp-devops-correction-docker`), à côté des `Dockerfiles`, pour que le déploiement continu s'intègre à GitHub Actions.

Étape 1 — Inventaire

J'ai créé un inventaire propre au projet dans `ansible/inventories/setup.yml` : il définit l'utilisateur SSH (`admin`), le chemin de la clé privée, et regroupe mon serveur sous le groupe `prod`.

```
all:
  vars:
    ansible_user: admin
    ansible_ssh_private_key_file: /Users/albantalagrand/.ssh/devops
  children:
    prod:
      hosts:
        alban.talagrand.takima.school:
```

Test de l'inventaire avec un ping (plus besoin de passer `--private-key` ni `-u` : tout est dans l'inventaire) :

```
ansible all -i inventories/setup.yml -m ping
```

3-1 — Inventaire et commandes de base

L'inventaire (`/etc/ansible/hosts` par défaut, ou ici un fichier projet) liste les serveurs gérés. Les [groupes] (ou `children`: en YAML) regroupent les hôtes par rôle (`webserver`, `db`, `proxy`...). Les variables (`ansible_user`, `ansible_ssh_private_key_file`) évitent de répéter les options sur chaque commande.

Commandes de base : - `ansible all -i inventories/setup.yml -m ping` → teste la connexion (pong).
 - `ansible all -i inventories/setup.yml -m setup -a "filter=ansible_distribution*" → récupère des facts (infos découvertes sur l'hôte).`
 - `ansible all -i inventories/setup.yml -m apt -a "name=apache2 state=absent" --become` → décrit l'état voulu (apache absent); Ansible converge le serveur vers cet état (idempotence : relancer ne change plus rien).

```
→ devops git:(main) × cd TP2/tp-devops-correction-docker/ansible
→ ansible git:(develop) × ansible all -i inventories/setup.yml -m ping
[WARNING]: Host 'alban.talagrand.takima.school' is using the discovered Python interpreter at
'/usr/bin/python3.11', but future installation of another Python interpreter could cause a dif
ferent interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.21/reference_
appendices/interpreter_discovery.html for more information.
alban.talagrand.takima.school | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.11"
  },
  "changed": false,
  "ping": "pong"
}
→ ansible git:(develop) ×
```

FIGURE 1 – Ping via inventaire

Étape 2 — Facts et nettoyage

`ansible_distribution*` me retourne les facts de l'hôte : Debian 12.7 (bookworm). J'ai ensuite supprimé Apache2 installé au TD3 (`state=absent`) → Ansible renvoie CHANGED la première fois, puis ok aux exécutions suivantes (idempotence).

Étape 3 & 4 — Playbook : installation de Docker

Un playbook décrit une suite de tâches à exécuter sur les hôtes. On peut le vérifier avant exécution avec `--syntax-check`.

```
ansible-playbook -i inventories/setup.yml playbook.yml --syntax-check
ansible-playbook -i inventories/setup.yml playbook.yml
```

`playbook.yml` installe Docker sur le serveur : prérequis apt, clé GPG + dépôt officiel Docker, `docker-ce`, puis un venv Python avec le SDK docker (nécessaire pour piloter les conteneurs depuis Ansible plus tard), et démarre le service Docker. `gather_facts: true` car le dépôt utilise `ansible_facts['distribution_release']` (= bookworm).

3-2 — Le playbook

C'est un fichier YAML qui cible des hôtes (`hosts: all`), s'exécute en root (`become: true`) et enchaîne des **tasks**, chacune appelant un module (`apt`, `apt_key`, `apt_repository`, `command`, `service`). Ansible est **idempotent** : il n'applique une tâche que si l'état réel diffère de l'état décrit (ex. `creates` : empêche de recréer le venv). On valide la syntaxe avec `--syntax-check`, puis on lance avec `ansible-playbook`.

Étape 5 — Refactor en rôle docker

J'ai déplacé l'installation de Docker dans un rôle dédié (`roles/docker`) pour un playbook plus propre et réutilisable :

```
ansible-galaxy init roles/docker # génère le squelette
```

Je n'ai gardé que les dossiers utiles : `tasks/` (les tâches d'installation) et `handlers/`. Le playbook se contente d'appeler le rôle :

```

→ ansible git:(develop) × ansible-playbook -i inventories/setup.yml playbook.yml
[WARNING]: Deprecation warnings can be disabled by setting 'deprecation_warnings=False' in ansible.cfg.
[DEPRECATION WARNING]: apt_key has been deprecated. Use deb822_repository instead. This feature will be removed from ansible-core 2.12. Origin: /Users/albantagrand/Dev/master/devops/TP2/tp-devops-correction-docker/ansible/playbook.yml:21:7

19     # Clé GPG officielle de Docker
20     - name: Add Docker GPG key
21       apt_key:
        ^ column 7

[DEPRECATION WARNING]: apt_repository has been deprecated. Use deb822_repository instead. This feature will be removed from ansible-core 2.12. Origin: /Users/albantagrand/Dev/master/devops/TP2/tp-devops-correction-docker/ansible/playbook.yml:27:7

25     # Dépôt APT stable de Docker
26     - name: Add Docker APT repository
27       apt_repository:
        ^ column 7

PLAY [all] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'alban.talagrand.takima.school' is using the discovered Python interpreter at '/usr/bin/python3.11', but future versions of Ansible will require a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.11/reference_appendices/interpreter_discovery.html for more information.
ok: [alban.talagrand.takima.school]

TASK [Install required packages] *****
changed: [alban.talagrand.takima.school]

TASK [Add Docker GPG key] *****
changed: [alban.talagrand.takima.school]

TASK [Add Docker APT repository] *****
changed: [alban.talagrand.takima.school]

TASK [Install Docker] *****
changed: [alban.talagrand.takima.school]

TASK [Install Python3 and pip3] *****
changed: [alban.talagrand.takima.school]

TASK [Create a virtual environment for Docker SDK] *****
changed: [alban.talagrand.takima.school]

TASK [Install Docker SDK for Python in virtual environment] *****
changed: [alban.talagrand.takima.school]

TASK [Make sure Docker is running] *****
ok: [alban.talagrand.takima.school]

PLAY RECAP *****
alban.talagrand.takima.school : ok=9    changed=7    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

FIGURE 2 – Playbook install Docker

```

→ ansible git:(develop) × ansible all -i inventories/setup.yml -m command -a "docker --version" --become
[WARNING]: Host 'alban.talagrand.takima.school' is using the discovered Python interpreter at '/usr/bin/python3.11', but future versions of Ansible will require a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.11/reference_appendices/interpreter_discovery.html for more information.
alban.talagrand.takima.school | CHANGED | rc=0 >>
Docker version 29.6.0, build fb59821

```

FIGURE 3 – Docker installé sur le serveur

```
- hosts: all
  gather_facts: true
  become: true
  roles:
    - docker
```

Je rejoue le playbook pour vérifier que le refactor fonctionne (idempotent : changed=0 si Docker est déjà installé) :

```
ansible-playbook -i inventories/setup.yml playbook.yml --syntax-check
ansible-playbook -i inventories/setup.yml playbook.yml
```

Étape 6 — Déploiement de l'application (rôles + docker_container)

J'ai créé un rôle par composant : network, database, app, proxy. Le playbook les enchaîne dans l'ordre (réseau d'abord, puis db, api, proxy). Les conteneurs sont lancés avec le module `community.docker.docker_container` (et `docker_network` pour le réseau), à partir des images publiées sur Docker Hub au TP2.

Prérequis sur le poste de contrôle (la collection `community.docker`) :

```
ansible-galaxy collection install community.docker
```

Les noms de conteneurs sont imposés par la résolution DNS Docker : - database → ciblé par l'API via `DATABASE_HOST=database`, - simple-api → ciblé par le proxy httpd (ProxyPass / http://simple-api:8080/), - httpd → exposé sur le port 80.

Point clé : les modules `docker_*` s'exécutent côté serveur et ont besoin du **SDK Python docker** installé dans le venv `/opt/docker_venv`. Le 2^e play force donc `ansible_python_interpreter: /opt/docker_venv/bin/python`, alors que le 1^{er} play (install via apt) garde l'interpréteur système.

```
- name: Run backend API
  community.docker.docker_container:
    name: simple-api
    image: "{{ dockerhub_user }}/tp-devops-simple-api:latest"
    pull: true
    networks:
      - name: "{{ network_name }}"
    env:
      DATABASE_HOST: database
      DATABASE_PASSWORD: "{{ postgres_password }}"
    state: started
    restart_policy: unless-stopped
```

3-3 — Configuration des tâches docker_container

Chaque tâche décrit l'état voulu d'un conteneur : `name` (nom DNS sur le réseau), `image` (depuis Docker Hub, via la variable `dockerhub_user`), `networks` (le réseau commun `app-network`), `env` (variables d'environnement injectées par Ansible — pas en dur dans l'image), `ports` (uniquement le proxy : 80:80), et `restart_policy: unless-stopped`. `pull: true` récupère la dernière image à chaque exécution (utile pour le déploiement continu). Ansible reste idempotent : si le conteneur tourne déjà avec la bonne config/image, il ne le recrée pas.

L'application est ensuite accessible via le serveur : `http://alban.talagrand.takima.school/departments/IRC/students`.

Étape 7 — Déploiement continu (GitHub Actions)

J'ai ajouté un job `deploy` au workflow `build-and-push.yml`, qui s'exécute **après** la publication des images (`needs: build-and-push-docker-image`) et seulement sur `main` (le workflow est déclenché par `workflow_run` après les tests). Le job installe Ansible + la collection `community.docker`, écrit la clé SSH depuis un secret GitHub (`ANSIBLE_SSH_KEY`) dans un fichier, puis rejoue le playbook.

```

→ ansible git:(develop) ✖ ansible-playbook -i inventories/setup.yml playbook.yml
[WARNING]: Deprecation warnings can be disabled by setting 'deprecation_warnings=False' in ansible.cfg.
[DEPRECATION WARNING]: apt_key has been deprecated. Use deb822_repository instead. This feature will be removed from ansible-core 2.16. Origin: /Users/albantagrand/Dev/master/devops/TP2/tp-devops-correction-docker/ansible/roles/docker/tasks/main.yml:17:3

15
16 - name: Add Docker GPG key
17   apt_key:
    ^ column 3

[DEPRECATION WARNING]: apt_repository has been deprecated. Use deb822_repository instead. This feature will be removed from ansible-core 2.16. Origin: /Users/albantagrand/Dev/master/devops/TP2/tp-devops-correction-docker/ansible/roles/docker/tasks/main.yml:22:3

20
21 - name: Add Docker APT repository
22   apt_repository:
    ^ column 3

PLAY [all] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'alban.talagrand.takima.school' is using the discovered Python interpreter at '/usr/bin/python3.11', but fallback to python3 found by searching for more information.
ok: [alban.talagrand.takima.school]

TASK [docker : Install required packages] *****
ok: [alban.talagrand.takima.school]

TASK [docker : Add Docker GPG key] *****
ok: [alban.talagrand.takima.school]

TASK [docker : Add Docker APT repository] *****
ok: [alban.talagrand.takima.school]

TASK [docker : Install Docker] *****
ok: [alban.talagrand.takima.school]

TASK [docker : Install Python3 and pip3] *****
ok: [alban.talagrand.takima.school]

TASK [docker : Create a virtual environment for Docker SDK] *****
ok: [alban.talagrand.takima.school]

TASK [docker : Install Docker SDK for Python in virtual environment] *****
changed: [alban.talagrand.takima.school]

TASK [docker : Make sure Docker is running] *****
ok: [alban.talagrand.takima.school]

PLAY [all] *****

TASK [network : Create docker network] *****
ok: [alban.talagrand.takima.school]

TASK [database : Run database] *****
ok: [alban.talagrand.takima.school]

TASK [app : Run backend API] *****
ok: [alban.talagrand.takima.school]

TASK [proxy : Run httpd proxy] *****
changed: [alban.talagrand.takima.school]

PLAY RECAP *****
alban.talagrand.takima.school : ok=13  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

```

FIGURE 4 – Déploiement via Ansible

```

→ ansible git:(develop) ✖ curl http://alban.talagrand.takima.school/departments/IRC/students
[{"id":1,"firstname":"Eli","lastname":"Copter","department":{"id":1,"name":"IRC"}}]

```

FIGURE 5 – API accessible via le serveur

```

deploy:
  needs: build-and-push-docker-image
  runs-on: ubuntu-24.04
  steps:
    - uses: actions/checkout@v4
    - name: Install Ansible
      run: |
        pip install ansible
        ansible-galaxy collection install community.docker
    - name: Write SSH private key
      run: |
        echo "${{ secrets.ANSIBLE_SSH_KEY }}" > ansible_key
        chmod 600 ansible_key
    - name: Run Ansible playbook
      working-directory: ./ansible
      env:
        ANSIBLE_HOST_KEY_CHECKING: "false"
      run: ansible-playbook -i inventories/setup.yml playbook.yml -e "ansible_ssh_private_key_file=${GITHUB_SSH_PRIVATE_KEY}"

```

Comme les conteneurs utilisent `pull: true`, chaque exécution récupère la dernière image : un push sur main déclenche test → build/push → déploiement automatique sur le serveur.

Question — Est-ce vraiment sûr de déployer automatiquement chaque nouvelle image ?

Non, pas totalement. Risques : toute image publiée part directement en production sans validation humaine ; le tag `latest` n'est pas déterministe (pas de version figée, donc rollback difficile) ; si les identifiants Docker Hub sont compromis, une image malveillante est déployée automatiquement (risque supply-chain) ; aucune garantie que l'image a passé les tests si quelqu'un pousse l'image à la main.

Pistes pour sécuriser : - ne déployer qu'après le passage des tests, et seulement sur main (déjà fait via `needs + workflow_run`) ; - **figer les tags** par SHA de commit au lieu de `latest` (traçabilité + rollback) ; - exiger une **validation manuelle** avant le déploiement prod (GitHub Environments + protection rules / required reviewers) ; - **scanner et signer** les images (Trivy, Docker Content Trust / cosign) ; - passer par un environnement de **staging** avant la prod ; - chiffrer les secrets avec un **Ansible Vault** et limiter les droits de la clé de déploiement (moindre privilège).

← Build and Push Docker Images

✓ Build and Push Docker Images #3

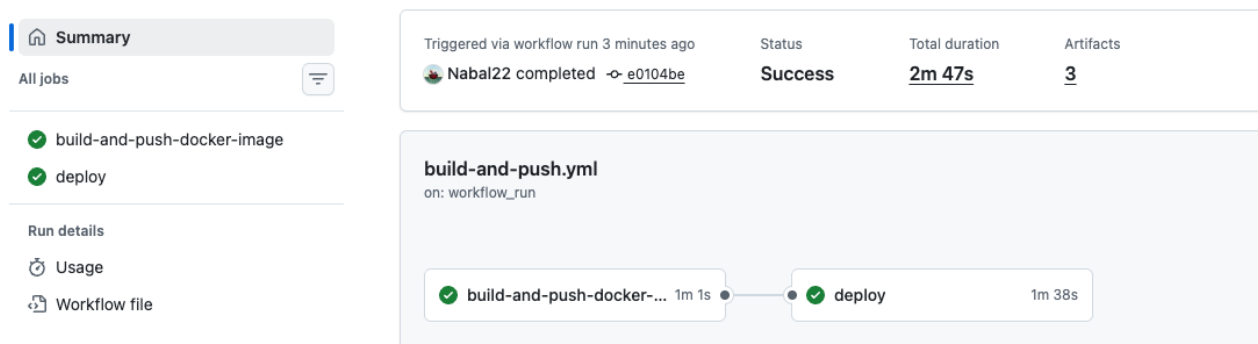


FIGURE 6 – Déploiement continu via GitHub Actions

Étape 8 — Front-end

J'ai ajouté le front (Vue.js, repo takima-training/devops-front) dans le dépôt sous front/. Le front est servi par nginx dans son propre conteneur, et le proxy httpd répartit les requêtes : - /api/... → backend simple-api:8080 (le préfixe /api est retiré), - / → conteneur front.

Le front appelle l'API via `http://${VUE_APP_API_URL}/...` J'ai donc fixé dans `front/.env.production` :

```
VUE_APP_API_URL=alban.talagrand.takima.school/api
```

Ainsi le navigateur appelle `http://alban.talagrand.takima.school/api/departments` → le proxy redirige vers `simple-api:8080/departments`. Tout passe par le port 80 (même origine, pas de problème CORS).

Conf du proxy (`http-server/my-httpd.conf`) :

```
<VirtualHost *:80>
ProxyPreserveHost On
ProxyPass /api/ http://simple-api:8080/
ProxyPassReverse /api/ http://simple-api:8080/
ProxyPass / http://front:80/
ProxyPassReverse / http://front:80/
</VirtualHost>
```

L'ordre compte : la règle la plus spécifique (/api/) doit être déclarée avant /.

J'ai créé un rôle front (conteneur front sur le réseau, sans port exposé — seul le proxy expose le 80) ajouté au playbook avant proxy, et ajouté la construction/push de l'image tp-devops-front dans le workflow CI. Le front est donc lui aussi déployé automatiquement.

Étape 9 — Aller plus loin : Ansible Vault

Pour ne plus stocker le mot de passe de la base en clair, j'ai séparé les variables en deux fichiers dans `group_vars/all/` : - `vars.yml` (en clair) : variables non sensibles + une référence `postgres_password` : `"{{ vault_postgres_password }}"`, - `vault.yml` (chiffré) : la vraie valeur `postgres_password` : `pwd`.

Le fichier `vault.yml` est chiffré avec **Ansible Vault** (AES) — il peut donc être commité sans risque. Création / édition :

```
ansible-vault create group_vars/all/vault.yml    # demande un mot de passe de vault
# contenu : vault_postgres_password: pwd
ansible-vault edit group_vars/all/vault.yml      # pour le modifier ensuite
```

Exécution du playbook en fournissant le mot de passe du vault :

```
ansible-playbook -i inventories/setup.yml playbook.yml --ask-vault-pass
```

En CI, le mot de passe du vault est stocké dans un secret GitHub (`ANSIBLE_VAULT_PASSWORD`), écrit dans un fichier et passé via `--vault-password-file`. Ainsi le déploiement continu déchiffre les secrets sans jamais les exposer.

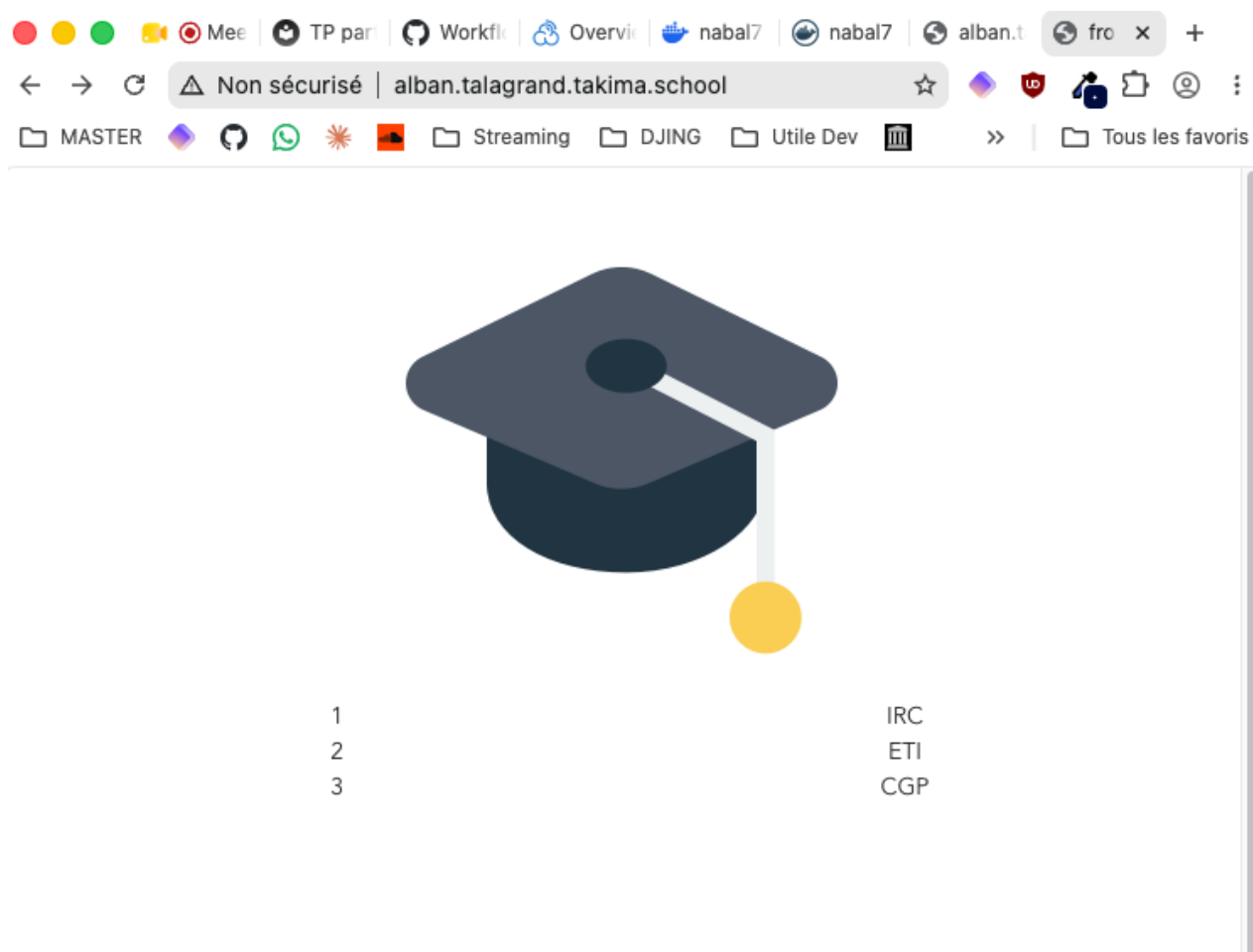


FIGURE 7 – Front accessible via le serveur

```
→ ansible git:(main) ✖ ansible-vault create group_vars/all/vault.yml
New Vault password:
Confirm New Vault password:
→ ansible git:(main) ✖ head -1 group_vars/all/vault.yml          # must show: $ANSIBLE_VAULT;1.1;AES256
$ANSIBLE_VAULT;1.1;AES256
```

FIGURE 8 – Fichier vault chiffré